

```

if (xContext == null) {
    System.err.println("ERROR: Could not bootstrap default Office");
    System.exit(0);
}

XMultiComponentFactory xMCF = xContext.getServiceManager();
Object desktop = xMCF.createInstanceWithContext("com.sun.star.frame.Desktop", xContext);

XComponentLoader xComponentLoader = (XComponentLoader)
UnoRuntime.queryInterface(XComponentLoader.class, desktop);

PropertyValue [] loadProps = new PropertyValue [0];
XComponent xComponent = xComponentLoader.loadComponentFrom
mURL("private:factory/scalc", "_blank", 0, loadProps);
} catch (java.lang.Exception e) {
    e.printStackTrace();
} finally {
    System.exit(0);
}

```

The packages required to import are not listed here; but NetBeans will help you for this purpose. You can add some more code to control spreadsheet(s) in the loaded Calc document, depending on your wishes.

Now use the **build** and **run** options of the project to test the extension. A new office instance will be created in which a blank Calc document is loaded. Note that the **deploy** option is not available for a client application because it is a simple J2SE application, as mentioned earlier.

This code looks very similar to the add-on code discussed in the previous article, except the first line where the context is already available as add-on code is invoked from the running instance.

Till now we have managed to create three types of projects without knowing much about UNO, so it is better to have a brief overview of UNO and some other terminology like API and IDL before going to the next example.

So do you know UNO?

Universal Network Objects (UNO) provide an environment to use OpenOffice.org services in a language-independent manner across platforms. It is a component model that, unlike others, is not bound to any programming language and offers interoperability between different languages.

UNO components can be created and accessed from any programming language, provided language binding is available for it. Languages like C++ and Java are well supported for UNO and a few more like Python or Ruby, which are under development.

The standalone environment of UNO objects that's isolated from OpenOffice.org is called the UNO Runtime Environment (URE).

OOo API

APIs are helpful to program OOo through UNO objects from many supported programming languages. The

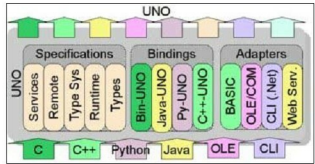


Figure 1: The UNO architecture [source udk.openoffice.org]

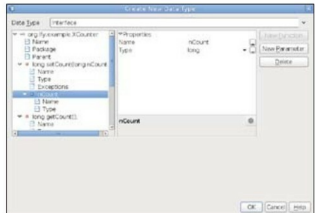


Figure 2: Creating an interface

main goal of the API project is to offer OOo as a service provider and integrate it with different applications.

IDL files

Interface Definition Language (IDL) files provide an abstract view of UNO objects. They contain attributes and methods. The IDL language used here is called UNO IDL.

A typical UNO IDL file looks like what follows:

```

#ifndef __org_ify_XCountable__
#define __org_ify_XCountable__
#include <com/sun/star/uno/XInterface.idl>
module org { module ify { module example {
    interface XCountable {
        long getCount();
        void setCount([in] long nCount);
    }; }; };
#endif

```

Every interface inherits from XInterface. Modules are similar to packages in Java or namespaces in C++. Attributes are supported by get/set methods. Compiled IDL files are then merged with the implementation part to create complete objects.